Section 1

1. What is Databricks primarily used for ?
   b) Data Engineering and analytics

2. Which feature in Databrciks helps track and manage the lineage of data transformation ?
   d) Unity Catalog

3. What is the primary methond for encrypting data at rest in Databricks?
   c) server side encryption

4. In Databricks, which type of cluster is specifically designed for development and ad-hoc queries ?
   d) All-purpose Cluster

5. Which Databricks feature allows for collaborative data science and data engineering across the full data lifecycle?
   a) Databricks Notebook

6. What is the primary purpose of Delta Lake in Databricks?

   b) To enhance data quality and reliablity

7. Which of the following services can Databricks integrate with for real-time data streaming ?
   b) Apache kafka

8. In Databricks, which is the function of the Databricks Runtime ?
   b) To provide a set of core libraries for data processing.

9. What is the Databricks SQL feature used for ?
   a) To build and run Spark SQL quries and dashboards

10. How can users track the performance and cost of their Databricks clusters?
    c) Cluster Metrics and Audit Logs

Section 3: Practical Exercises

1. Cluster Configuration for Notebook Development:
    a. Describe the steps to create a cluster suitable for developing notebooks in Databricks.

1. Create a New Cluster

1. Log in to your Databricks workspace.
2. Navigate to **Compute** → **Create Compute**.
3. Configure the cluster:
    - **Cluster Name:** MongoDB-SomeDelta-Notebook-Cluster
    - **Cluster Mode:** Single Node (for development) or Standard (for team usage)
    - **Databricks Runtime:** Latest LTS version (e.g., DBR 14.x or higher)
    - **Node Type:** Small/Medium instance depending on data volume
    - **Autoscaling:** Optional (Min 1, Max 2 workers for development)

2. Install Required Libraries

For MongoDB connectivity, install the MongoDB Spark Connector.

Maven Coordinate:

org.mongodb.spark:mongo-spark-connector_2.12:10.3.0

Go to:

- Cluster → Libraries → Install New → Maven
- Paste the coordinate and install.

3. Configure MongoDB Connection

Add Spark configurations in the cluster or notebook:

```
spark.conf.set(
  "spark.mongodb.read.connection.uri",
  "mongodb://username:password@host:27017/database.collection"
)
```

4. Create a Notebook

1. Workspace → Create → Notebook
2. Select Python as the language.
3. Attach the notebook to the created cluster.

b. The notebook should read data from a MongoDB and write it to a Delta table, processing 100 records at a time.

```
(
spark.read
 .format("mongodb")
.option("database", "salesdb") // database config
.option("collection", "orders") //  collection config
.load() // load data from mongodb
.repartition(100) // 100 records per partition
.write // write it into the Delta table
.format("delta")
.mode("append")
.saveAsTable("bronze.mongo_orders")
)
```

2. Python Code for Data Processing:
    a. Write a Python script in Databricks to read data from the “customers” collection in MongoDB. The structure of the data includes:
        i. ID (integer)
        ii. Customer Name (string)
        iii. City (string)
        iv. Age (integer)
    b. Identify the oldest customer in each city and write the result to a Delta table located at /mnt/delta/sample_data.
    c. Use the following Databricks secrets for the MongoDB conection:
        i. Mongo Connection String:
            1. Scope: mongodb
            2. Key: src_conn
    d. Database Name:
        i. Scope: mongodb
        ii. Key: database

3. Data Extraction by Data Range:
    a. Extend the previous script to extract data from the MongoDB “customers” collection between two dates, which should be provided as parameters.
    b. The script should support execution in both development and production environments, using the following Databricks secrets:
        i. Development Environment:

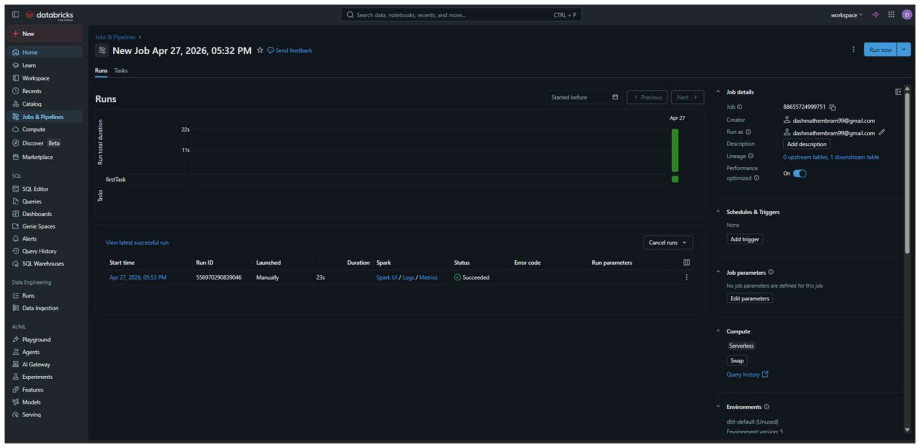
1. Scope: mondodb.dev
2. Key: src_conn (connection string)
3. Key: database (database name)

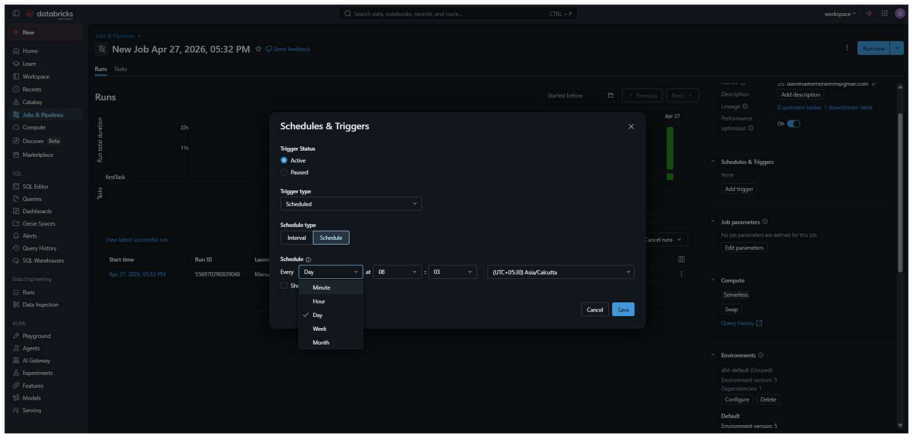
ii. Production Environment:

1. Scope: mongodb.prod
2. Key: src_conn (connection string)
3. Key: database (database name)

Note: Ensure that the Customer_Name field is encrypted before writing to the Delta location /mnt/delta/sample_data, with a mechanism to decrypt using a secret key.

4. Scheduling and Notifications
   a. Describe the steps to schedule the notebook from the previous exercise to run daily using Databricks features.





Steps to Schedule the Notebook Daily

1. **Create and Test the Notebook**
   - Ensure the notebook that reads from MongoDB and writes to the Delta table executes successfully.
   - Save the notebook in the workspace.

2. **Create a Job**
    - In Databricks, navigate to **Workflows → Jobs**.
    - Click **Create Job**.
3. **Configure the Task**
    - Provide a job name, e.g., MongoDB_To_Delta_Daily_Load.
    - Add a task:
        - **Task Type:** Notebook
        - Select the notebook created in the previous exercise.
        - Choose an existing cluster or configure a job cluster.
4. **Set the Schedule**
    - Under **Schedule**, click **Add Trigger**.
    - Select **Scheduled**.
    - Configure:
        - Frequency: **Daily**
        - Time: e.g., **01:00 AM**
        - Time Zone: as per business requirements (e.g., IST).
5. **Save and Run**
    - Click **Create**.
    - Run the job manually once to validate execution.
    - Databricks will automatically execute the notebook daily according to the schedule.

b. Set up a notification system to send an email in case of successful execution.

In above config we could add

Configure Notifications

- Add email recipients for:
    - Job success
    - Job failure
    - Duration warnings

5. Databricks cluster Creation from WSL Terminal:
    a. Detail the process of creating a Databricks cluster using the Databricks CLI on a Window Subsystem for Linux (WSL) terminal.

    To create a Databricks cluster using the Databricks CLI on WSL, install and configure the CLI with a Personal Access Token (PAT), create a cluster configuration JSON file, and execute the cluster creation command. The cluster status can then be verified using Databricks CLI commands.

6. Implement a Spark SQL query
    a. Write a Spark SQL query in Databricks to calculate the average, minimum, and maximum age of cuctomer in each city from the MongoDB “custoimers” collection. Store the results in a Delta table located at /mnt/delta/customer_age_stats.
7. Data Transformaytion and Cleaning
    a. Create a notebook that reads data from CSV file located at /mnt/data/raw/customers_csv, cleanses the data (e.g. handles missing values, standardizes formats), and writes the cleaned data to a Delta table at /mnt/delta/cleaned_custmers. Document the transformations applied.
8. Automated Testing and CI/CD Pipeline
    a. Describe how you would set up automated testing for databricks notebooks and integrate them into a CI/CD pipeline. Include considerations for testing different environment (e.g. dev , staging , prod) and ensuring code quality.

        1. Version Control
            a. Commit notebook and source code to feature branch.
            b. Connect Databricks to a Git repo. i.e GitHub, GitLab
        2. Environment Management
            a. Store env-specific config seperatly
            b. dev:
                mongo_uri=dev_db
                delta_path=/dev/data

                staging:
                mongo_uri=staging_db
                delta_path=/staging/data

                prod:
                mongo_uri=prod_db
                delta_path=/prod/data
            c. should avoid gardcodeing creds
        3. Implement Automated Testing
            a. Using testing framework such as PyTest to test py code
            b. Mock external db system (MongoDB, API’s)
            c. Create test dataset to validate transformation
        4. Code Quality Control
            a. Code review view Pull request
            b. Linting
            c. Formatting check